

CSCE 2014 - Laboratory Assignment 9

The purpose of this lab is to introduce students to the **queue** abstract data type and show how this ADT can be used to implement discrete event simulations.

1. Introduction to Queues

The queue abstract data type is used to store and access data in a FIFO (first in first out) manner. For example if we store 3 values A, B, and C in a queue. Then when we retrieve the data, we will get A, B, and C. The 'A' was the first value stored, so it will be the first value retrieved.

Queues have found a number of uses in computer systems. Perhaps the most important use of queues is in operating systems, where they are used to schedule the execution multiple tasks that are sharing the CPU. Each task gets to run for a short amount of time, and then it must go to the end of the execution queue and wait its turn to run again.

Five basic operations can be performed on a queue:

Remove: To retrieve and remove data from the front of the queue

Insert: To store an item at the end of the queue

Front: To retrieve the item from the front of the queue

IsEmpty: Check whether the queue is empty

IsFull: Check whether the queue is full

2. The Queue Class

Consider the C++ declarations below for a Queue class. Notice that we are storing integer values in the queue, and that the length of the queue is fixed at compile time.

```
//-----  
// Purpose:   Header file for the Queue class.  
// Author:    John Gauch  
//-----  
#include <cmath>  
#include <iostream>  
#include <iomanip>  
using namespace std;  
  
const int QUEUE_SIZE = 10;  
  
//-----  
// Define the queue class interface.  
//-----  
class Queue  
{  
public:  
    // constructor functions  
    Queue();  
    ~Queue();
```

```

    // general queue operations
    bool IsFull();
    bool IsEmpty();
    void Insert(int Number);
    void Remove(int &Number);
    int Remove();
    int GetLength();
    void Print();

private:
    int Data[QUEUE_SIZE];
    int End;
};

```

3. Queue Implementation

Consider the implementation of the Queue class below. Notice that this implementation uses an array to store the queue data and it shifts the data to left as data is removed from the queue. This will be slower than using a circular queue, but it is easier to visualize the queue contents.

```

//-----
// Purpose:    Implementation of Queue class.
// Author:     John Gauch
//-----
#include "queue.h"

//-----
Queue::Queue()
{
    End = -1;
    for (int i = 0; i < QUEUE_SIZE; i++)
        Data[i] = -1;
}

//-----
Queue::~Queue()
{
}

//-----
bool Queue::IsFull()
{
    return (End >= QUEUE_SIZE-1);
}

//-----
bool Queue::IsEmpty()
{
    return (End < 0);
}

//-----
void Queue::Insert(int item)
{
    // check if full

```

```

        if (IsFull())
            cout << "queue overflow\n";

        else
            // insert data
            Data[++End] = item;
    }

//-----
void Queue::Remove(int &item)
{
    // check if empty
    if (IsEmpty())
        cout << "queue underflow\n";

    else
    {
        // remove data
        item = Data[0];

        // shift data
        for (int i = 0; i < End; i++)
            Data[i] = Data[i + 1];
        Data[End--] = -1;
    }
}

//-----
int Queue::Remove()
{
    int item;
    Remove(item);
    return item;
}

int Queue::GetLength()
{
    return (End+1);
}

//-----
void Queue::Print()
{
    cout << End + 1 << " : ";
    for (int i = 0; i <= End; i++)
        cout << setw(2) << Data[i] << " ";
    cout << endl;
}

//-----
// Purpose:    Test the Queue class.
// Author:     John Gauch
//-----
#include "queue.h"

int main()
{

```

```

Queue q;
q.Insert(3);
q.Insert(1);
q.Insert(4);
q.Print();
q.Insert(1);
q.Insert(5);
q.Insert(9);
q.Print();

int num;
q.Remove(num);
q.Remove(num);
q.Print();
q.Remove(num);
q.Remove(num);
q.Print();
q.Remove(num);
q.Remove(num);
q.Print();
}

```

4. Testing the Queue Class

Copy and paste the code above into three files "queue.h", "queue.cpp" and "main.cpp" in your IDE. Now run your program to see what it prints. Edit the "queue.h" file and change the size of the queue to see if you can get the code to print some error messages.

Cut and paste your modified program output below:

```

3: 3 1 4
queue overflow
6: 3 1 4 1 5 9
4: 4 1 5 9
2: 5 9
0:

```

5. More Testing of Queue Class

You might be worried that the queue implementation above will become slow if there is a lot of data being stored. To test this hypothesis, modify the "queue.h" file to make the queue size 100 long, and modify the main function in "queue.cpp" to insert and remove 100 random values between 0..9 into the queue.

Hint: If you use "int value = rand() % range;" the variable value will contain a random number between [0..range-1]. When you run your program, roughly how long does it take? A few seconds? Several minutes? Try bumping the number up to 1000. Now how long does it take?

Cut and paste your modified main program below:

```

int main() {
    Queue q;
    for (int i = 0; i < 1000; i++) {
        if (!q.IsFull())
            q.Insert(rand() % 10);
    }
    q.Print();

    for (int i = 0; i < 1000; i++) {
        if (!q.IsEmpty())
            q.Remove();
    }
    q.Print();
}

```

6. Discrete Event Simulation

As noted above, queues are often used to implement discrete event simulations. The idea is to keep track of when events occur, and simulate what happens when processing an event. For example, in a store, the arrival of a customer is the "customer arrival event". If the store has one waiting line and several clerks, the customer is put into a queue until one of the clerks is available. When the customer gets to the front of the queue, there is a "customer service event". Finally, when the customer leaves the store, we have a "customer departure event".

If we want to get fancy, we can model how long each customer transaction will take, and the average speed of different clerks. We can also add "customer loss events" to simulate frustrated customers who give up and go home if the wait is too long. After the simulation, we could decide it is time to hire more clerks, or change the way different transactions are processed.

```

//-----
// Purpose:      This program demonstrates how multiple queues
//               can be used to simulate customer arrivals.
// Author:       John Gauch
//-----
#include "queue.h"

//-----
// Main program uses the Queue class to simulate customers.
//-----
int main()
{
    int NumLines = 4;
    int NumCustomers = 20;
    int CurrentTime = 0;
    Queue *Vendor = new Queue[NumLines];

    // Add customers to different queues
    for (int Customer = 0; Customer < NumCustomers; Customer++)
    {
        // Generate customer data
        CurrentTime += 5 + random() % 20;
        int NumItems = 1 + random() % 5;

        // Search for shortest queue
        int ShortLine = 0;
        int ShortLength = Vendor[0].GetLength();
        for (int Line = 1; Line < NumLines; Line++)
        {
            int Length = Vendor[Line].GetLength();

```

```

        if (Length < ShortLength)
        {
            ShortLine = Line;
            ShortLength = Length;
        }
    }

    // Add customer to shortest line
    Vendor[ShortLine].Insert(NumItems);
}

// Print all customer lines
for (int Line = 0; Line < NumLines; Line++)
{
    cout << "Line " << Line << " ";
    Vendor[Line].Print();
}
cout << endl;
}

```

Consider the simple discrete event simulation above. This program has a fixed number of customers and a fixed number of queues. Notice that customers always get into the shortest line. Copy the code above into "main.cpp" in your IDE and run the program. The program should print the queues after all the customers have arrived. Now, edit the main function to **change the number of customers and queues**. In order to see the simulation better move some code to **print the queue contents after each queue insertion**.

Cut and paste your program output below:

```

Line 0 1: 2
Line 1 1: 1
Line 2 1: 1
Line 0 2: 2 3
Line 1 2: 1 2
Line 2 2: 1 3
Line 0 3: 2 3 5
Line 1 3: 1 2 2
Line 2 3: 1 3 2
Line 0 4: 2 3 5 2

Line 0 4: 2 3 5 2
Line 1 3: 1 2 2
Line 2 3: 1 3 2

```

7. Improving the Event Simulation

As you have noticed, the customers in this simulation never leave the queues. To correct this, we can check all of the queues when a customer arrives, to see if one or more of the current customers are finished and ready to leave. For this simulation, we can simply assume that there is a 20% probability that they are finished. Thus, if we generate a random number between [0..99] and the value is between [0..19] then we remove the customer from the front of the queue. If not, they stay there until the next customer comes in.

Step 1: Modify the main function of the simulation to add this logic to the main loop in the simulation, and test it out with different numbers of customers and queues. This should make things look slightly more realistic. In general, what happens as we add queues to the simulation?

Step 2: Notice that even with this logic, there are still customers in the queues after the last customer arrives. To fix this, you need to add another loop to remove customers from queues after the last customer has arrived. This loop should continue removing customers at random until all of the queues are empty.

Cut and paste your modified main program below:

```

#include "queue.h"

//-----
// Main program uses the Queue class to simulate customers.
//-----
int main() {
    int NumLines = 3;
    int NumCustomers = 10;
    int CurrentTime = 0;
    Queue *Vendor = new Queue[NumLines];

    // Add customers to different queues
    for (int Customer = 0; Customer < NumCustomers; Customer++) {
        // Generate customer data
        CurrentTime += 5 + random() % 20;
        int NumItems = 1 + random() % 5;

        // Search for shortest queue
        int ShortLine = 0;
        int ShortLength = Vendor[0].GetLength();
        for (int Line = 1; Line < NumLines; Line++) {
            int Length = Vendor[Line].GetLength();
            if (Length < ShortLength) {
                ShortLine = Line;
                ShortLength = Length;
            }
        }

        // Add customer to shortest line
        Vendor[ShortLine].Insert(NumItems);

        // Check each line for a departure with 20% probability
        for (int Line = 0; Line < NumLines; Line++) {
            if (!Vendor[Line].IsEmpty() && (random() % 100 < 20))
                Vendor[Line].Remove();
        }

        // Print all customer lines after each arrival.
        cout << "Time " << CurrentTime << endl;
        for (int Line = 0; Line < NumLines; Line++) {
            cout << "Line " << " ";
            Vendor[Line].Print();
        }
        cout << endl;
    }

    cout << "Begin drain\n..." << endl;
    bool Empty = false;
    while (!Empty) {
        Empty = true;
        for (int Line = 0; Line < NumLines; Line++) {
            if (!Vendor[Line].IsEmpty()) {
                Empty = false;
                if (random() % 100 < 20)
                    Vendor[Line].Remove();
                // cout << "Line " << " ";
                // Vendor[Line].Print();
            }
        }
    }

    // Print final customer lines.
    cout << "Final" << endl;
    for (int Line = 0; Line < NumLines; Line++) {
        cout << "Line " << Line << " ";
        Vendor[Line].Print();
    }

    delete[] Vendor;
    cout << endl;
}

```

Cut and paste your program output below:

Line 1: 2

Line 0:

Line 0:

Time 28

Line 1: 2

Line 1: 4

Line 0:

Time 45

Line 1: 2

Line 1: 4

Line 1: 5

Time 60

Line 2: 2 5

Line 1: 4

Line 1: 5

Time 71

Line 2: 2 5

Line 1: 3

Line 1: 5

Time 83

Line 2: 2 5

Line 2: 3 5

Line 1: 5

Time 91

Line 2: 2 5

Line 2: 3 5

Line 1: 3

Time 98

Line 2: 2 5

Line 2: 3 5

Line 2: 3 4

Time 119

Line 3: 2 5 2

Line 2: 3 5

Line 2: 3 4

Time 125

Line 3: 2 5 2

Line 3: 3 5 5

Line 2: 3 4

Begin drain

Line 3: 2 5 2

Line 2: 5 5

Line 2: 3 4

Line 2: 5 2

Line 2: 5 5

Line 2: 3 4

Line 2: 5 2

Line 2: 5 5

Line 2: 3 4

Line 2: 5 2

Line 2: 5 5

Line 2: 3 4

Line 2: 5 2

Line 1: 5

Line 2: 3 4

Line 2: 5 2

Line 1: 5

Line 2: 3 4

Line 1: 2

Line 1: 5

Line 2: 3 4

Line 0:

Line 1: 5

Line 2: 3 4

Line 1: 5

Line 2: 3 4

Line 1: 5

Line 1: 4

Line 1: 5

Line 1: 4

```
Line 1: 4
Line 1: 5
Line 1: 4
Line 1: 5
Line 1: 4
Line 0:
Line 1: 4
Line 1: 4
Line 1: 4
Line 1: 4
Line 0:
```

```
Final
Line 0 0:
Line 1 0:
Line 2 0:
```



8. Submit Work

When you have completed all of the steps to this lab:

- At the bottom of the page enter your name and UAID and click "Submit Lab".
- Print the resulting lab report web page to a PDF file.
- Login to Blackboard and go to the labs section.
- Upload your PDF file for grading and feedback.

Enter your FIRST name:

Enter your LAST name:

Enter your UAID number: